

A Project Report on
“EXPENSE TRACKER WITH SMART INSIGHTER”

Submitted in partial fulfilment of the requirements
For the award of the degree of Bachelor of Technology in Computer Science
and Engineering



MP-ONLINE LIMITED
B.Tech CSE (AI&DS) 6th Semester

Submitted by
Anubhav Mishra (B2355R10195121)

Under the Guidance of
Mr. Tulsiram Vishwakarma

Department of Computer Science & Engineering
AKS University, Satna (M.P.)

ABSTRACT

The "**Expense Tracker with Smart Insider**" is a full-stack web application designed to help users efficiently manage and analyse their daily financial activities. Traditional expense tracking systems often lack intelligent insights and require manual categorisation, which limits their effectiveness in real-life financial decision-making. This project addresses these limitations by integrating smart features such as automated expense categorisation, predictive spending analysis, and anomaly detection. The system allows users to record expenses, set monthly budgets, and visualise their spending patterns through interactive dashboards and charts. A rule-based Natural Language Processing (NLP) approach is used to automatically categorise expenses based on user input descriptions. Additionally, the system provides predictive insights by estimating monthly spending and generating alerts when users are likely to exceed their budget. Anomaly detection is implemented to identify unusual transactions, helping users maintain better financial control. The application is developed using React for the frontend, FastAPI for the backend, and PostgreSQL as the database. Data visualisation is implemented using charting libraries to provide meaningful insights in a user-friendly manner.

Overall, "**Expense Tracker with Smart Insider**" ultimately serves as a personal financial assistant, empowering users to take control of their money and achieve financial discipline. By combining ease of use with insightful analytics, the project contributes to better financial awareness and planning, helping individuals save more and spend wisely. This project demonstrates the integration of full-stack development with basic AI techniques to create a practical and intelligent financial management system.

TABLE OF CONTENTS

•	Introduction.....	4-5
•	Objective of The Project.....	6-7
•	Scope of The Projects	8-9
•	System Overview.....	10-11
•	Technology Stack.....	12-14
•	System Architecture	15-19
•	System Configuration	20-22
•	Database Design	23-26
•	Implementation	27-29
•	Features.....	30-31
•	ER Diagram	31-32
•	DFD Diagram	32-33
•	Block Diagram.....	33-34
•	Testing and Validation	35-38
•	Result/ Output Screens	39-44
•	Conclusion	44-45

INTRODUCTION

In today's fast-paced digital world, effective financial management has become essential for individuals to maintain control over their expenses and savings. With the increasing use of online transactions and digital payments, users often find it difficult to track their daily spending habits manually. As a result, there is a growing need for intelligent systems that not only record expenses but also provide meaningful insights into spending behaviour. Traditional expense tracking applications primarily focus on recording transactions and generating basic reports. However, they often lack advanced analytical capabilities such as automatic categorisation, predictive analysis, and detection of unusual spending patterns. This limitation makes it difficult for users to make informed financial decisions. The "Expense Tracker with Smart Insighter" project aims to overcome these challenges by developing a smart financial management system that combines expense tracking with intelligent insights. The application allows users to record their daily expenses, categorise them automatically using a rule-based Natural Language Processing (NLP) approach, and visualise their spending patterns through interactive charts and dashboards.

One of the primary motivations behind developing this project is the growing awareness of financial literacy. Many people find it difficult to manage their money effectively, leading to overspending, debt accumulation, and poor savings habits. This project aims to serve as a practical tool that not only records transactions but also educates users about their spending patterns. By providing visual representations such as charts and graphs, the system helps users identify areas where they can cut costs and improve their financial discipline.

In addition to basic tracking features, the system provides predictive insights by estimating monthly spending based on user behaviour. It also includes a budget management feature that allows users to set a monthly spending limit and receive alerts when they are likely to exceed it. Furthermore, anomaly detection is implemented to identify unusual transactions that deviate significantly from normal spending patterns. The system is developed using modern web technologies, including React for the frontend, FastAPI for the backend, and PostgreSQL for database management. The integration of these technologies ensures a responsive user interface, efficient data handling, and scalable architecture.

Another significant aspect of the project is its potential for scalability. While the current version focuses on individual users, the system can be expanded to support families, small businesses, or even organisations that require expense management solutions. Features such as multi-user access, shared budgets, and collaborative tracking could transform the Expense Tracker into a versatile financial management platform. In addition to its practical benefits, the Expense Tracker project serves as an educational exercise in software development. It demonstrates the application of concepts such as database management, user interface design, and data visualisation. For students and developers, the project provides valuable experience in building real-world applications that solve everyday problems.

Overall, this project focuses on transforming a simple expense tracker into an intelligent financial assistant that helps users understand, control, and optimise their spending habits.

OBJECTIVE OF THE PROJECT

The main objectives of the “Expense Tracker with Smart Insighter” project are as follows:

- 1. To develop a user-friendly web application for tracking daily expenses efficiently:** This objective focuses on providing a simple and intuitive interface where users can easily add, view, and manage their daily financial transactions without complexity.
- 2. To automate the categorisation of expenses using a rule-based Natural Language Processing approach:** The system automatically assigns categories such as Food, Transport, or Shopping based on the description entered by the user, reducing manual effort.
- 3. To provide visual insights into spending patterns through interactive charts and dashboards:** Graphical representations like pie charts and line charts help users understand how their money is being spent across different categories.
- 4. To enable users to set and manage a monthly budget for better financial control:** Users can define a budget limit, which helps them monitor and control their overall spending effectively.
- 5. To implement predictive analysis for estimating monthly spending based on user behaviour:** The system analyses past expense data to predict future spending trends, helping users plan their finances in advance.
- 6. To generate alerts when the user is likely to exceed the defined budget limit:** Warning messages are displayed when spending approaches or exceeds the budget, encouraging responsible financial behaviour.
- 7. To detect unusual or abnormal transactions using anomaly detection techniques:** The system identifies transactions that significantly deviate from normal spending patterns and highlights them for user attention.
- 8. To ensure secure access through user authentication and session management:** The application includes login and registration features to protect user data and ensure only authorised access.

9. To design a scalable and maintainable system using modern full-stack technologies:

The use of technologies like React, FastAPI, and PostgreSQL ensures that the system is efficient, modular, and easy to maintain.

10. To enhance decision-making by converting raw expense data into meaningful insights:

The overall goal is to help users make better financial decisions by providing clear, data-driven insights rather than just raw numbers.

SCOPE OF THE PROJECT

The scope of the “Expense Tracker with Smart Insider” project is centred on developing an intelligent and user-friendly financial management system that helps individuals monitor, analyse, and optimise their daily expenses. The system is designed to transform basic expense tracking into a more insightful and decision-supporting process by integrating analytical and predictive features. The application enables users to record their daily financial transactions in a structured manner. It further enhances usability by automatically categorising expenses based on the description provided by the user using a rule-based Natural Language Processing (NLP) approach. This reduces manual effort and improves consistency in expense classification.

The major functionalities included within the scope of this project are:

- Recording, updating, and deleting expense entries (CRUD operations).
- Automatic categorisation of expenses into predefined categories such as Food, Transport, Shopping, and Bills.
- Visualisation of financial data using interactive charts such as pie charts and line graphs.
- Monthly budget setting and tracking to help users manage their financial limits.
- Prediction of monthly expenditure based on current spending behaviour.
- Generation of alerts when the predicted spending exceeds the defined budget.
- Detection of unusual or abnormal transactions using basic anomaly detection techniques.
- Secure user authentication through login and registration functionality.

The system is primarily intended for students and individual users who require a simple yet intelligent solution for managing their personal finances. Being a web-based application, it ensures accessibility across multiple devices without the need for installation.

However, the current scope of the project is limited to manually entered expense data and basic rule-based intelligence. It does not include integration with banking systems, real-time transaction synchronisation, or advanced machine learning algorithms. Additionally, the security mechanisms implemented are suitable for a prototype-level system and may require further enhancement for production-level deployment.

In terms of future expansion, the system can be enhanced by incorporating real-time bank API integration, advanced machine learning models for improved prediction accuracy, personalised financial recommendations, voice-based input features, and mobile application support.

Overall, the scope of this project focuses on delivering a functional, scalable, and intelligent expense tracking system while leaving room for future technological enhancements and improvements.

SYSTEM OVERVIEW

The “Expense Tracker with Smart Insighter” is a full-stack web-based application designed to provide an efficient and intelligent solution for managing personal finances. The system not only enables users to record and track expenses but also transforms financial data into meaningful insights through analytics and basic AI-driven features.

System Type:

The application is a client-server-based web system that follows a modular and scalable design. It separates the frontend, backend, and database layers to ensure better maintainability, flexibility, and future extensibility.

Architecture:

The system is built using a three-tier architecture consisting of:

1. Presentation Layer (Frontend)

2. Application Layer (Backend)

3. Data Layer (Database)

❖ Frontend (Presentation Layer):

The frontend is developed using React, which provides a dynamic, responsive, and user-friendly interface. It allows users to perform key operations such as registration, login, adding and managing expenses, setting monthly budgets, and viewing financial insights through charts and dashboards. The interface is designed to be simple and intuitive, making it accessible even for users with minimal technical knowledge.

❖ Backend (Application Layer):

The backend is implemented using FastAPI, which handles all business logic and API operations. It manages user authentication, processes expense-related requests, and acts as an intermediary between the frontend and the database. The backend is responsible for performing CRUD (Create, Read, Update, Delete) operations on expenses and integrating intelligent features such as automatic categorisation, prediction, and anomaly detection.

❖ Database (Data Layer):

PostgreSQL is used as the database management system to store structured data. It maintains user details and expense records, including attributes such as amount, description, category, and date. The database ensures data consistency, efficient querying, and reliable storage, which is essential for generating accurate analytics and reports.

❖ AI and Insight Module:

The system incorporates a basic AI component to enhance user experience. A rule-based Natural Language Processing (NLP) approach is used to automatically categorise expenses based on keywords in the user-provided descriptions. Additionally, predictive analysis is implemented to estimate monthly spending patterns, and anomaly detection logic is used to identify unusual or high-value transactions that deviate from normal behaviour.

❖ System Workflow:

The overall workflow begins with user authentication through the login and registration system. Once authenticated, users can add expense data via the frontend interface. This data is sent to the backend through API calls, where it is processed, categorised using AI logic, and stored in the PostgreSQL database. The stored data is then retrieved and displayed on the dashboard, where users can view insights such as total spending, category-wise distribution, budget usage, and spending trends using graphical visualisations like pie charts and line charts.

The system also allows users to set a monthly budget and provides real-time feedback in the form of alerts and insights when spending approaches or exceeds the defined limit. This helps users make better financial decisions and maintain control over their expenses.

Overall, the integration of modern web technologies with intelligent data processing ensures that the system not only functions as a basic expense tracker but also acts as a smart financial assistant. It provides a scalable foundation that can be extended in the future with advanced machine learning models, real-time data integration, and mobile platform support.

TECHNOLOGY STACK

The “Expense Tracker with Smart Insighter” is developed using a carefully selected full-stack technology combination that ensures performance, scalability, and maintainability while keeping the development process efficient for rapid prototyping.

1. Frontend Layer:

- ❖ The frontend of the application is built using React.js, a component-based JavaScript library that enables the development of dynamic and reusable user interface elements. React’s virtual DOM mechanism improves rendering performance by updating only the necessary components instead of reloading the entire page. This results in a faster and smoother user experience.
- ❖ React is used to manage multiple views, such as Login, Registration, and Dashboard, through client-side routing. State management is handled using React Hooks, allowing efficient handling of user input, API responses, and UI updates. The frontend communicates with the backend through REST API calls using Axios, ensuring seamless data exchange.

2. Backend Layer:

- ❖ The backend is developed using FastAPI, a high-performance Python web framework designed for building APIs quickly and efficiently. FastAPI is chosen due to its asynchronous capabilities, automatic validation using Pydantic models, and built-in support for API documentation (Swagger UI).
- ❖ The backend is responsible for:
 1. Handling user authentication (login and registration).
 2. Processing expense-related operations (create, update, delete, fetch).
 3. Implementing business logic such as budget management and prediction.
 4. Integrating AI-based categorisation logic.
 5. FastAPI ensures low latency and high throughput, making it suitable for scalable backend systems.

3. Database Layer:

- ❖ PostgreSQL is used as the relational database management system for storing application data. It provides strong data consistency, ACID compliance, and efficient query processing.
- ❖ The database stores structured data in tables such as:
 - Users (user credentials and profile details).
 - Expenses (amount, description, category, date).

SQL queries are used to perform CRUD operations, and indexing ensures faster data retrieval for dashboard analytics.

4. AI and Intelligence Layer:

- ❖ The system incorporates a lightweight AI component implemented in Python. Instead of complex machine learning models, a rule-based Natural Language Processing (NLP) approach is used for expense categorisation. This method analyses keywords in the transaction description and assigns categories such as Food, Transport, Shopping, etc.
- ❖ In addition to categorisation, predictive logic is implemented to estimate monthly spending based on user input patterns. Anomaly detection is performed using statistical comparison techniques, where transactions significantly deviating from average spending are flagged as unusual.
- ❖ This approach ensures that the system delivers intelligent insights while remaining computationally efficient and easy to maintain.

5. Data Visualisation Layer:

To present financial insights effectively, the application uses Recharts, a charting library for React. It is used to generate:

- Pie charts for category-wise expense distribution.

These visualisations help users quickly interpret their financial data and make informed decisions.

6. Development and Version Control Tools:

- ❖ The project is developed using Visual Studio Code, which provides a flexible development environment with support for extensions, debugging tools, and integrated terminal access.
- ❖ Version control is managed using Git, and the project repository is hosted on GitHub. This enables efficient tracking of code changes, version management, and collaboration.

Overall, the chosen technology stack ensures a modular architecture where each component performs a specific role efficiently. The combination of modern frontend, high-performance backend, structured database, and intelligent logic layer results in a robust and scalable application.

SYSTEM ARCHITECTURE

The **Expense Tracker with Smart Insider** is designed using a structured, layered architecture that promotes clarity, scalability, and long-term maintainability. Instead of building the application as a single tightly coupled system, it is divided into multiple logical layers, where each layer performs a specific role and interacts with others through well-defined interfaces. This design approach ensures that the system remains flexible, easier to debug, and capable of handling future enhancements such as advanced analytics or AI integrations. The architecture is broadly divided into four primary layers: Presentation Layer (Frontend), Application Layer (Backend), Data Layer (Database), and Intelligence Layer (AI/Logic). Together, these layers create a seamless flow of data and functionality across the system.

1. Presentation Layer (Frontend):

The Presentation Layer represents the user-facing part of the application and is implemented using **React.js**. It is responsible for capturing user inputs, presenting processed data in a meaningful format, and ensuring an intuitive and responsive user experience. This layer is structured using reusable components, which improves code organisation and allows efficient updates or feature additions. Key components include authentication interfaces such as login and registration pages, a centralised dashboard for viewing expense summaries, forms for adding and editing expense records, and graphical elements such as charts for data visualisation.

One of the main strengths of using React is its ability to update the user interface dynamically without reloading the entire page. This is achieved through state management and component-based rendering, which ensures a smooth and interactive experience. From a communication perspective, the frontend does not directly interact with the database. Instead, it sends HTTP requests to the backend through RESTful APIs. These requests include operations such as creating a new expense, retrieving expense history, or authenticating a user. The responses received from the backend are then rendered in the UI.

Additionally, the frontend is responsible for basic input validation, improving user experience by preventing invalid data submission before it reaches the backend.

2. Application Layer (Backend):

The Application Layer forms the backbone of the system and is implemented using **FastAPI**, a modern and high-performance Python web framework. This layer acts as an intermediary between the frontend and the database while also coordinating with the intelligence layer. The backend is designed using a modular structure, where different functionalities are separated into distinct modules such as routing, services, and models. This modularisation ensures better code readability and simplifies debugging and testing.

Key responsibilities of this layer include:

- Handling incoming API requests from the frontend.
- Authenticating users and managing secure access.
- Validating incoming data using schemas.
- Executing business logic related to expense tracking.
- Performing CRUD operations on the database.
- Integrating AI-based logic for smart features.

FastAPI provides automatic request validation using Pydantic models, which ensures that only properly structured data is processed. It also generates interactive API documentation, which is useful during development and testing. Security is an important aspect handled by this layer. User credentials are securely managed, and sensitive data such as passwords is stored in encrypted form. The backend also ensures proper error handling, returning meaningful responses in case of invalid operations or system failures.

3. Data Layer (Database):

The Data Layer is responsible for persistent storage of all application data and is implemented using **PostgreSQL**, a robust relational database management system. The database is structured into well-defined tables to ensure data integrity, consistency, and efficient querying. The primary tables include:

- **Users Table:** Stores user-related information such as name, email, and hashed password
- **Expenses Table:** Stores transaction details including amount, category, description, and timestamp

Relationships between tables are defined to maintain referential integrity. For example, each expense record is linked to a specific user, ensuring that data remains organised and user-specific. The backend interacts with the database using an ORM (Object Relational Mapping) approach, which simplifies database operations by allowing developers to work with Python objects instead of raw SQL queries. This improves development speed and reduces the chances of errors.

The database layer ensures:

- Reliable storage of financial data
- Efficient retrieval of large datasets
- Consistency through transactional operations

4. Intelligence Layer (AI/Logic):

The Intelligence Layer is a key differentiator of this project, transforming it from a basic expense tracker into a smart financial assistant. This layer is implemented using Python-based logic integrated within the backend. It focuses on enhancing user experience by providing intelligent insights rather than just storing data.

Core functionalities of this layer include:

a. Automatic Expense Categorisation: The system uses rule-based Natural Language Processing techniques to analyse the description of each expense. Based on predefined keywords, it automatically assigns a relevant category such as food, travel, or utilities. This reduces manual effort for users and improves consistency in data classification.

b. Predictive Analysis: The system estimates future spending patterns using historical data. It calculates averages and trends to provide users with an idea of their expected monthly expenses. This feature helps users plan their budgets more effectively.

c. Anomaly Detection: The system identifies unusual transactions by comparing current expenses with past spending behaviour. If a transaction significantly deviates from normal patterns, it is flagged as an anomaly. This helps users quickly notice unexpected or potentially problematic expenses. This layer is designed to be extensible, meaning more advanced machine learning models can be integrated in the future without major architectural changes.

5. Data Flow and System Interaction:

The interaction between different layers follows a well-defined and continuous data flow:

1. The user interacts with the frontend interface, such as entering a new expense.
2. The frontend sends an HTTP request to the backend API.
3. The backend validates the request and processes it using business logic.
4. The intelligence layer analyses the data for categorization and insights.
5. The processed data is stored in the PostgreSQL database.
6. When required, the backend retrieves updated data from the database.
7. The response is sent back to the frontend.
8. The frontend updates the user interface dynamically using the received data.

This structured flow ensures consistency, real-time updates, and a smooth user experience across the application.

6. Architectural Benefits:

The layered architecture offers several significant advantages:

- **Separation of Concerns:** Each layer operates independently, making the system easier to understand and manage.
- **Scalability:** New features, such as advanced AI models or third-party integrations, can be added without affecting existing components.
- **Maintainability:** Modular design allows developers to update or fix specific parts without impacting the entire system.
- **Flexibility:** The system can be extended to support mobile applications or cloud deployment.
- **Performance Optimisation:** Each layer can be optimised independently based on system requirements.

The system architecture of the Expense Tracker with Smart Insighter is designed to be robust, modular, and future-ready. By combining a responsive frontend, a powerful backend, a reliable database, and an intelligent logic layer, the application not only fulfils its core purpose of expense tracking but also delivers meaningful insights to users. This architecture lays a strong foundation for future enhancements, such as advanced machine learning models, real-time analytics, and scalable cloud-based deployment, making the system adaptable to evolving user needs and technological advancements.

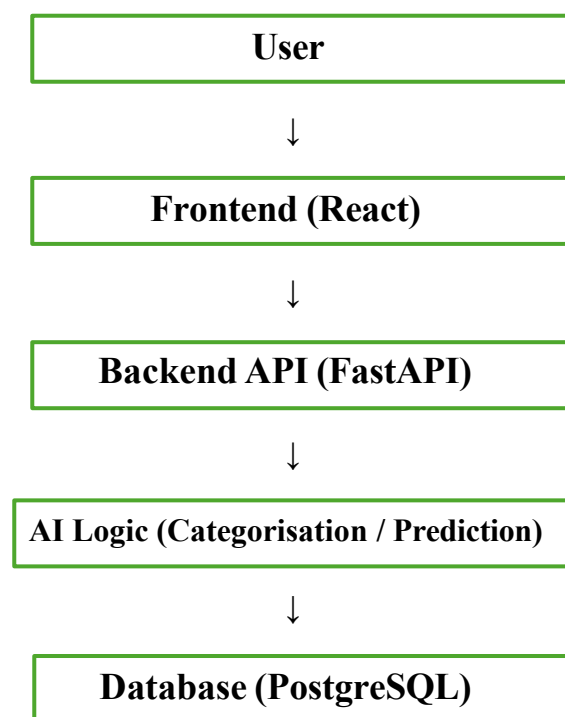


Figure 1. System Architecture Diagram

SYSTEM CONFIGURATION

1. Hardware Requirements:

- **Processor:** Minimum Intel Core i3 or equivalent; recommended Intel Core i5/i7 for faster performance.
- **RAM:** Minimum 4 GB; recommended 8 GB or higher for handling AI analysis and data visualisation.
- **Storage:** At least 500 MB of free disk space for application files and local database; recommended 2 GB or more for storing historical expense records.
- **Display:** Standard resolution (1366x768) supported; recommended Full HD (1920x1080) for better visualization of charts and dashboards.
- **Connectivity:** Internet connection required for cloud synchronisation, AI updates, and notifications.

2. Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu preferred for open-source setups).
- **Programming Language:** Python 3.9 or higher (core language for backend, AI, and visualisation).
- **Database:** SQLite (lightweight local storage) or MySQL/PostgreSQL for scalable setups.
- **Web Frameworks:** Flask or Django for backend API development.
- **Frontend Tools:** HTML, CSS, JavaScript (with optional frameworks like React/Angular for advanced UI).
- **Visualisation Libraries:** Matplotlib, Seaborn, Plotly for generating charts and graphs.

- **AI/ML Libraries:** Scikit-learn, TensorFlow, or PyTorch for implementing savings suggestions.

3. Development Environment:

- **IDE/Editor:** Visual Studio Code (recommended for Python development with extensions for debugging and linting).
- **Version Control:** Git/GitHub for source code management and collaboration.
- **Package Manager:** pip (Python package installer) for managing dependencies.
- **Virtual Environment:** venv or Anaconda for isolating project dependencies.
- **Testing Tools:** PyTest or unittest for validating modules and ensuring system reliability.

4. System Workflow Configuration:

1. **User Authentication:** Secure login system configured with hashed passwords.
2. **Income Input:** Monthly income stored in the database and linked to budget calculations.
3. **Expense Recording:** Daily expenses logged with category and date, stored in structured tables.
4. **AI Analysis:** Configured to run periodically, analysing spending patterns and generating tips.
5. **Visualisation:** Charts and reports generated dynamically from stored data.
6. **Alerts:** Notification system configured to trigger when expenses exceed budget thresholds.
7. **Backup:** Database backup scheduled weekly to prevent data loss.

5. Security Configuration:

- **Encryption:** Sensitive data (user credentials, financial records) encrypted using SHA-256 or AES.
- **Authentication:** Role-based access control to prevent unauthorised use.
- **Data Privacy:** Compliance with standard practices to ensure user data is not exposed.
- **Backup & Recovery:** Automated backup scripts configured for restoring data in case of failure.

6. Scalability Configuration:

- **Cloud Integration:** Optional setup with AWS, Azure, or Google Cloud for multi-device access.
- **Load Handling:** Database optimised with indexing for faster queries.
- **Future Expansion:** Configured to support predictive analytics, family accounts, or business expense tracking.

DATABASE DESIGN

The database design of the **Expense Tracker with Smart Insider** is built on a relational model using **PostgreSQL**, ensuring structured storage, strong data integrity, and efficient retrieval of financial information. Since the application deals with user-specific financial data, the design focuses on maintaining accuracy, consistency, and secure data handling while supporting smooth interaction with the backend services. The database is intentionally kept simple and normalised in its initial stage, with a clear separation of user data and transaction data. This approach avoids redundancy, improves performance, and makes the system easier to scale in the future.

At its core, the database consists of two primary tables: **Users** and **Expenses**, with a well-defined relationship between them.

1. Users Table:

The **Users** table is responsible for storing all registered user information. It acts as the foundation for authentication and user-specific data management within the system.

Each user is uniquely identified by a system-generated primary key, ensuring that every record can be accessed and managed independently.

Attributes of Users Table:

- **id**: Integer, Primary Key, Auto-increment Uniquely identifies each user in the system.
- **Name**: Text (Stores the full name of the user).
- **email**: Text, Unique Acts as the primary identifier for login and authentication.
- **password**: Text Stores the hashed version of the user's password for security.

The use of a unique constraint on the email field ensures that no two users can register with the same email address. Additionally, passwords are never stored in plain text, which strengthens the overall security of the system.

2. Expenses Table:

The **Expenses** table stores all financial transactions entered by users. This table is central to the application's functionality, as it holds the data used for tracking, analysis, and insights.

Each expense record is linked to a specific user, ensuring that data remains personalised and isolated between different users.

Attributes of Expenses Table:

- **id**: Integer, Primary Key, Auto-increment Uniquely identifies each expense record.
- **user_id**: Integer, Foreign Key (references Users.id) Links the expense to a specific user.
- **amount**: Float Represents the monetary value of the expense.
- **description**: Text Stores additional details about the transaction.
- **category**: Text Stores the category assigned through AI-based or rule-based logic.
- **date**: Date represents the date on which the expense occurred.

The inclusion of the category field allows the system to support intelligent features such as categorisation and analytics. The date field enables time-based analysis, such as monthly spending trends.

3. Relationship Between Tables:

The relationship between the **Users** and **Expenses** tables is defined as a **one-to-many relationship**, which is a common and effective design for such applications.

- One user can have multiple expense records.
- Each expense record belongs to exactly one user.

This relationship is implemented using a foreign key constraint on the `user_id` field in the Expenses table, referencing the primary key of the Users table.

This design ensures:

- Logical grouping of data per user.
- Easy retrieval of user-specific expenses.
- Strong data consistency across tables.

4. Data Integrity and Constraints:

Maintaining data integrity is a critical aspect of the database design. Several constraints are applied to ensure that the data stored is accurate, consistent, and reliable.

Key constraints include:

- **Primary Key Constraints:** Ensure that each record in both tables is uniquely identifiable.
- **Foreign Key Constraint:** Maintains referential integrity between Users and Expenses tables, preventing orphan records.
- **NOT NULL Constraints:** Applied to essential fields such as `user_id`, `amount`, and `email` to ensure required data is always present.
- **Unique Constraint on Email:** Prevents duplicate user registrations and ensures reliable authentication.
- **Data Validation Rules:** Although primarily handled at the backend level, the database structure supports validation by enforcing correct data types.

These constraints collectively ensure that the system maintains high data quality and avoids inconsistencies.

5. Database Operations and Query Handling:

The application performs several types of database operations to support its functionality. These operations are executed through the backend using ORM tools, ensuring clean and efficient interaction with the database.

Common operations include:

- **Insert Operations:** used to add new users during registration and new expense records.
- **Select Queries:** Retrieve user-specific expense data for dashboards, reports, and analytics.
- **Update Operations:** Allow users to modify existing expense records.
- **Delete Operations:** Enable removal of unwanted or incorrect expense entries.

These operations are optimized to ensure fast response times and accurate data retrieval, even as the dataset grows.

6. Performance and Scalability Considerations:

The database design is built with scalability in mind, allowing the system to grow without major structural changes.

Some important considerations include:

- **Normalization:** Reduces redundancy and improves data consistency.
- **Indexing (Future Scope):** Indexes can be added on frequently queried fields such as `user_id` and `date` to improve performance.
- **Extensibility:** New tables can be introduced in the future, such as:
 - Categories table for dynamic category management.
 - Budget tracking table.
 - Transaction history logs.
 - AI insights or prediction storage.
- **Handling Large Data Volumes:** PostgreSQL is capable of efficiently managing large datasets, making it suitable for long-term use.

The database design of the Expense Tracker with Smart Insighter provides a strong foundation for storing and managing financial data. By using a relational structure with clearly defined tables, relationships, and constraints, the system ensures data integrity, efficient querying, and secure storage. At the same time, the design remains flexible and scalable, allowing the application to evolve with additional features such as advanced analytics, AI-driven insights, and expanded financial management capabilities.

IMPLEMENTATION DETAILS

The implementation of the **Expense Tracker with Smart Insighter** follows a structured and modular development approach, where each component of the system is built, tested, and integrated step by step. The goal of this approach is to ensure clarity in development, ease of debugging, and smooth integration between different parts of the application. The system combines frontend, backend, database, and intelligent logic into a single cohesive application. Each module is designed independently but works together to deliver a seamless user experience.

1. User Authentication Implementation:

User authentication is one of the foundational components of the system. It ensures that only registered users can access and manage their financial data. During the **registration process**, the application collects user details such as name, email, and password through the frontend interface. This data is sent to the backend, where it is validated and securely stored in the database. Passwords are not stored in plain text; instead, they are processed using hashing techniques to enhance security. During **login**, the system verifies the entered credentials against the stored data. If the credentials are valid, the user is successfully authenticated and granted access to the dashboard. To maintain the login state, a lightweight session mechanism is implemented on the frontend using local storage. This allows the application to remember the user session and restrict access to protected routes unless the user is authenticated.

2. Expense Management (CRUD Operations):

The core functionality of the system revolves around managing expenses. This is implemented using standard CRUD (Create, Read, Update, Delete) operations.

- **Create:** Users can add a new expense by entering details such as amount and description.
- **Read:** All stored expenses are retrieved from the backend and displayed on the dashboard.
- **Update:** Users can modify existing expense entries through an edit feature.
- **Delete:** Users can remove any unwanted or incorrect expense records.

These operations are handled through dedicated API endpoints in the backend. The frontend triggers these operations using HTTP requests, ensuring smooth interaction between the user interface and the server.

3. API Integration:

The communication between the frontend and backend is handled through **RESTful APIs**, forming a clear and structured interaction layer. Whenever a user performs an action, such as adding an expense or retrieving data, the frontend sends an HTTP request to the backend. The backend processes the request, applies necessary logic, interacts with the database, and sends back a response in JSON format.

This separation between frontend and backend ensures:

- Loose coupling between components.
- Easier debugging and testing.
- Flexibility to scale or replace components independently.

API integration is a key part of making the system responsive and maintainable.

4. AI-Based Expense Categorisation:

To enhance usability, the system includes an automatic expense categorisation feature. This is implemented using a rule-based Natural Language Processing approach. When a user enters an expense description, the backend analyses the text and matches keywords against predefined categories such as food, travel, shopping, and utilities. Based on this analysis, the system assigns an appropriate category to the expense. This logic is implemented in a separate service module, ensuring that it remains independent from core business logic. This separation improves code organisation and allows future upgrades, such as integrating machine learning models.

5. Budget Management and Alerts:

The system allows users to define a monthly budget, adding a layer of financial planning to the application. Once a budget is set, the system continuously tracks total expenses and compares them against the defined limit. If the system detects that the predicted monthly spending may exceed the budget, it generates an alert. This feature helps users stay aware of their spending habits and take corrective actions before exceeding their financial limits.

6. Predictive Analysis:

A simple predictive analysis mechanism is implemented to estimate future spending. The system calculates the average of past expenses and uses it to project expected monthly expenditure. Although this approach is basic, it provides valuable insights into spending patterns and helps users plan their finances more effectively. The design allows for future improvement by integrating more advanced predictive models.

7. Anomaly Detection:

The system includes a basic anomaly detection feature to identify unusual or unexpected transactions. Each expense is compared with the user's typical spending behaviour. If a transaction significantly deviates from the average, it is flagged as an anomaly. This allows users to quickly identify high-value or irregular expenses. This feature adds an additional layer of awareness and helps users maintain better control over their finances.

8. Data Visualisation:

To make the data more meaningful and easier to understand, the system includes visual representations of expenses.

The dashboard uses charting libraries to display:

- **Pie charts** for category-wise expense distribution.

These visual elements help users quickly interpret their financial data without going through raw numbers, improving overall usability.

9. User Interface and Experience:

The user interface is designed with simplicity and usability in mind. The layout is clean, and navigation between different sections is smooth and intuitive. Forms are designed to be responsive and easy to use, ensuring that users can quickly add or update expenses. Basic input validation is implemented at the frontend level to reduce errors and improve the overall experience. Routing is handled efficiently to allow seamless movement between pages, such as login, dashboard, and expense management.

FEATURES

The "Expense Tracker with Smart Insighter" provides a combination of basic expense management functionalities along with intelligent features to enhance user experience and decision-making. The key features of the system are as follows:

1. User Authentication:

The system allows users to securely register and log in using their credentials. This ensures that each user's data is private and protected.

2. Expense Management:

Users can add, view, update, and delete their expenses. This complete set of CRUD operations enables efficient management of financial records.

3. Automatic Expense Categorisation:

The system automatically categorises expenses based on the description entered by the user using a rule-based NLP approach. This reduces manual effort and improves accuracy.

4. Budget Management:

Users can set a monthly budget and monitor their spending against it. This helps in maintaining financial discipline.

5. Predictive Spending Analysis:

The system estimates the user's monthly spending based on current expense patterns. This predictive insight helps users plan their finances better.

6. Overspending Alerts:

The application generates warnings when the predicted spending exceeds the defined budget. This feature helps users take corrective action in time.

7. Anomaly Detection:

Unusual or high-value transactions are identified and highlighted. This allows users to quickly detect unexpected expenses.

8. Data Visualisation:

The system provides visual insights through charts such as:

- Pie chart for category-wise expense distribution
- Line chart for spending trends

These visualisations make it easier to understand financial data.

9. Interactive Dashboard:

All important information, including expenses, insights, and alerts, is displayed in a single dashboard for easy access.

10. Responsive User Interface:

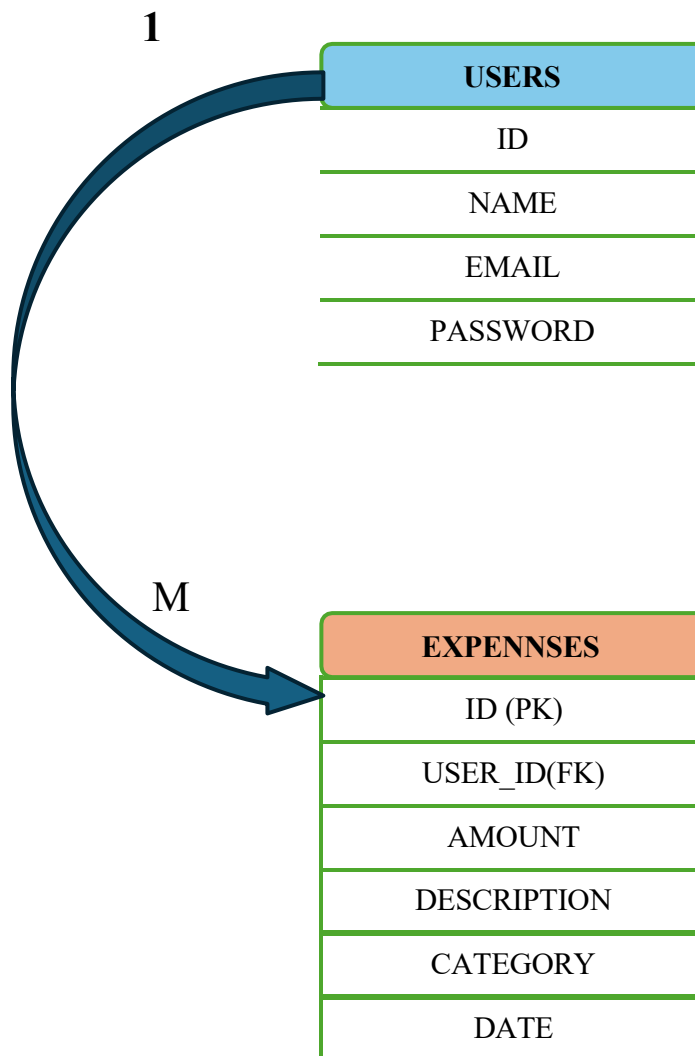
The application provides a clean and user-friendly interface that works smoothly across different devices.

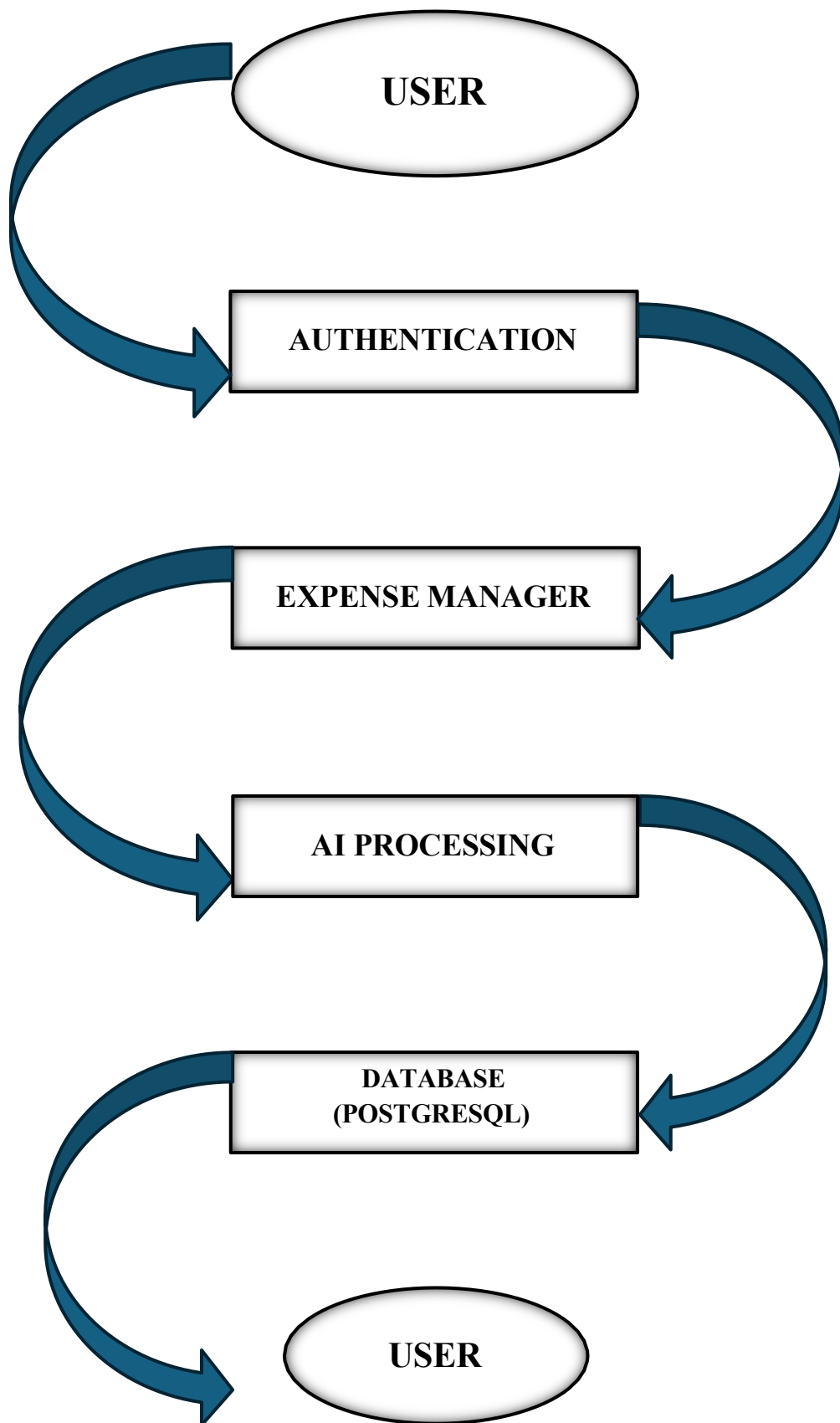
11. Edit and Delete Functionality:

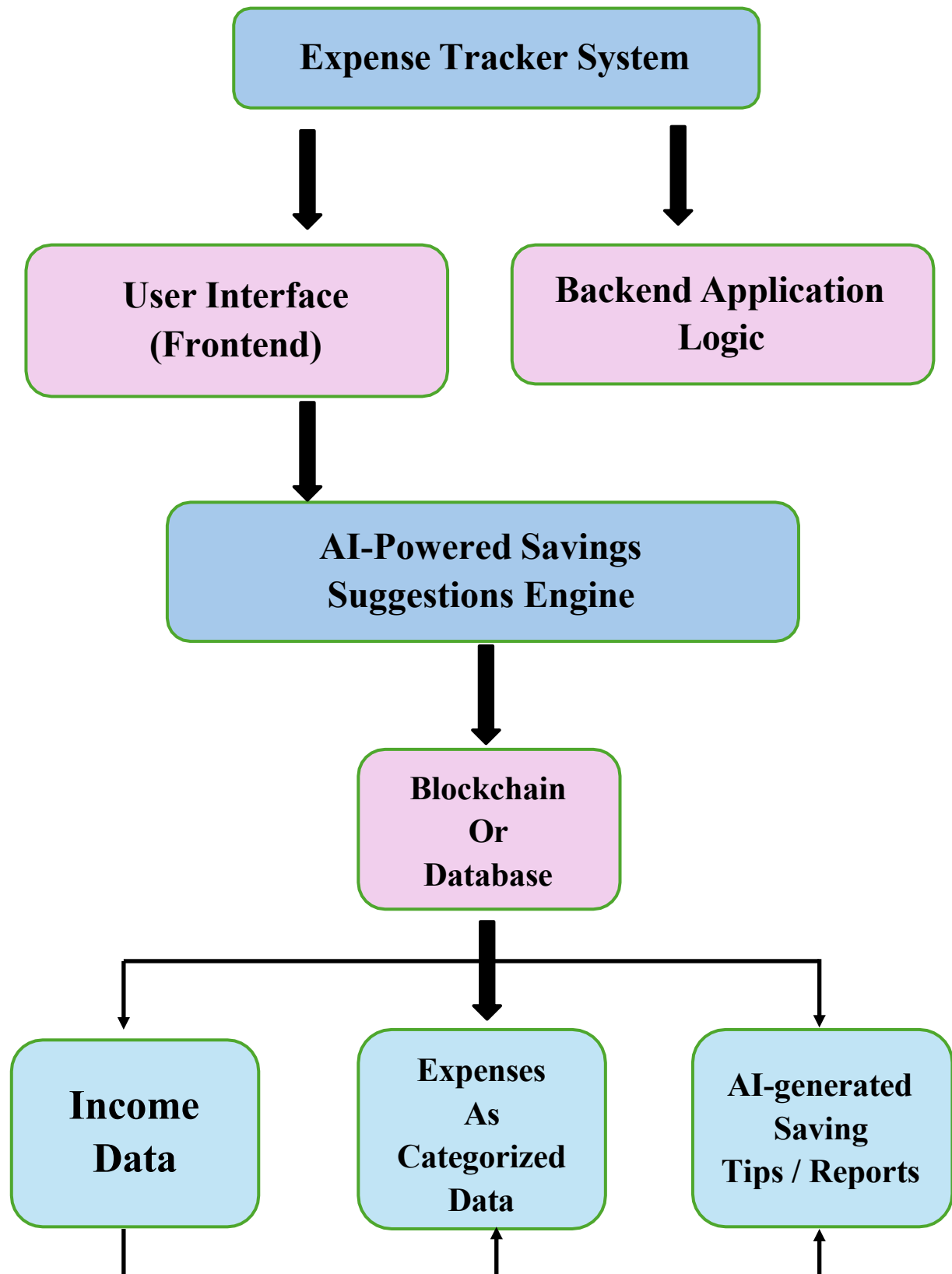
Users can easily modify or remove existing expenses using intuitive controls, improving flexibility and usability.

Overall, the system combines essential expense tracking features with intelligent analysis tools to deliver a comprehensive financial management solution.

ENTITY RELATIONSHIP DIAGRAM



DATA FLOW DIAGRAM

BLOCK DIAGRAM

TESTING AND VALIDATION

Testing plays a critical role in the development of the **Expense Tracker with Smart Insider**, ensuring that the application performs as expected across different scenarios and usage conditions. A structured testing approach was followed to verify the correctness, reliability, and efficiency of each module within the system. The testing process focused not only on identifying defects but also on improving overall system quality and user experience.

Multiple levels of testing were carried out, covering individual components, system integration, functionality, and usability.

1. Unit Testing:

Unit testing was performed to validate the smallest functional components of the system in isolation. The goal was to ensure that each function behaves correctly before integrating it with other modules.

In the backend, individual API endpoints were tested to verify operations such as adding, updating, retrieving, and deleting expenses. Similarly, specific logic components, including expense categorization and validation functions, were tested independently.

On the frontend side, key functions responsible for handling user inputs, API calls, and state updates were verified. This approach helped in identifying errors at an early stage and ensured that each unit of the system works as intended.

2. Integration Testing:

After validating individual components, integration testing was carried out to ensure smooth communication between different layers of the application.

This involved testing the interaction between:

- React frontend and FastAPI backend through REST APIs
- Backend and PostgreSQL database for data storage and retrieval

API requests sent from the frontend were tested to confirm that the backend correctly processes them and returns accurate responses. Additionally, database operations were verified to ensure that data is properly stored and retrieved without inconsistencies.

This phase ensured that all components work together as a unified system.

3. Functional Testing:

Functional testing was conducted to verify that the system meets all specified requirements and delivers the expected features.

The following core functionalities were tested:

- User registration and login process
- Adding, editing, and deleting expense records
- Automatic expense categorization based on descriptions
- Budget setting and alert generation
- Display of data through charts and dashboard components

Each feature was tested under normal and edge-case scenarios to ensure consistent and correct behaviour. This type of testing ensured that the application fulfills its intended purpose effectively.

4. Validation Testing:

Validation testing focused on ensuring that the system properly handles user input and prevents invalid data from entering the system.

Key validation checks included:

- Preventing submission of empty fields
- Verifying correct email format during registration
- Enforcing minimum password requirements
- Ensuring numeric fields such as amount accept only valid values

Both frontend and backend validations were tested to provide an additional layer of reliability. This reduces the chances of incorrect data affecting system performance or analysis.

5. Performance Testing:

Basic performance testing was carried out to evaluate how efficiently the system responds to user actions.

The application was tested with multiple expense entries to observe:

- Response time of API calls
- Speed of data retrieval from the database
- Rendering performance of charts and dashboard

The system demonstrated stable performance under normal usage conditions, with smooth data updates and minimal delay in UI rendering.

6. Usability Testing:

Usability testing was performed to assess how easily users can interact with the system. The focus was on ensuring that the interface is intuitive, simple, and easy to navigate.

Key aspects evaluated included:

- Clarity of navigation between pages
- Simplicity of forms and input fields
- Readability of displayed data and charts
- Overall user experience during common tasks

The results confirmed that users can perform actions such as adding expenses or viewing insights without confusion, making the application user-friendly.

7. Error Handling and Exception Testing:

The system was also tested for its ability to handle unexpected scenarios and errors gracefully.

Test cases included:

- Invalid login attempts
- Incorrect API requests
- Server-side failures or missing data

Appropriate error messages are displayed to users, guiding them to take corrective actions.

This improves the robustness of the system and enhances user trust.

Test Table

Test Case ID	Feature	Input	Expected Output	Result
TC01	Login	Valid email & password	User logged in successfully	Passed
TC02	Login	Invalid email or password	Error message displayed	Passed
TC03	Registration	Valid name, email, password	User registered successfully	Passed
TC04	Registration	Existing email	Error: Email already exists	Passed
TC05	Add Expense	Amount = 500, Desc = "Pizza"	Expense added, category = Food	Passed

Test Case ID	Feature	Input	Expected Output	Result
TC06	Add Expense	Empty fields	Validation error shown	Passed
TC07	Edit Expense	Update amount/description	Expense updated successfully	Passed
TC08	Delete Expense	Click the delete button	Expense removed from the list	Passed
TC09	Categorization	Desc = "Uber ride"	Category = Transport	Passed
TC10	Budget Setting	Budget = 10000	Budget saved and displayed	Passed
TC11	Overspending Alert	Spending exceeds the budget	Warning message displayed	Passed
TC12	Prediction	Multiple expenses added	Monthly prediction shown	Passed
TC13	Anomaly Detection	High-value expense (e.g., 5000)	Expense flagged as unusual	Passed
TC14	Pie Chart	Categorised expenses available	Category-wise chart displayed	Passed
TC15	Logout	Click logout	User redirected to login page	Passed
TC16	Route Protection	Access dashboard without login	Redirected to the login page	Passed

RESULTS / OUTPUT SCREENS

This section presents the key user interface screens of the **Expense Tracker with Smart Insider**, highlighting how different functionalities are implemented and experienced by the user. Each screen reflects a specific feature of the system and demonstrates how users interact with the application in real-world usage.

1. Login Page:

The Login Page serves as the entry point for registered users. It allows users to securely access their accounts by entering their email and password.

The interface is designed to be simple and user-friendly, with clearly labelled input fields. Basic validation is implemented to ensure that users provide the required information before submitting the form. In case of incorrect credentials, appropriate error messages are displayed to guide the user.

This page ensures secure authentication and prevents unauthorised access to the system.

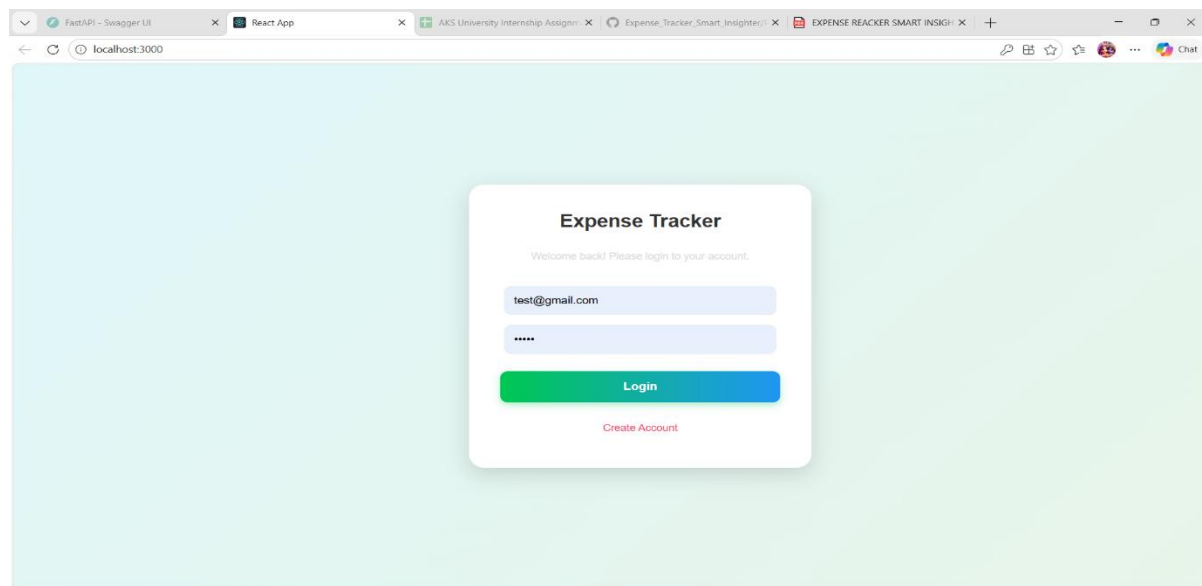


Figure : Login Page

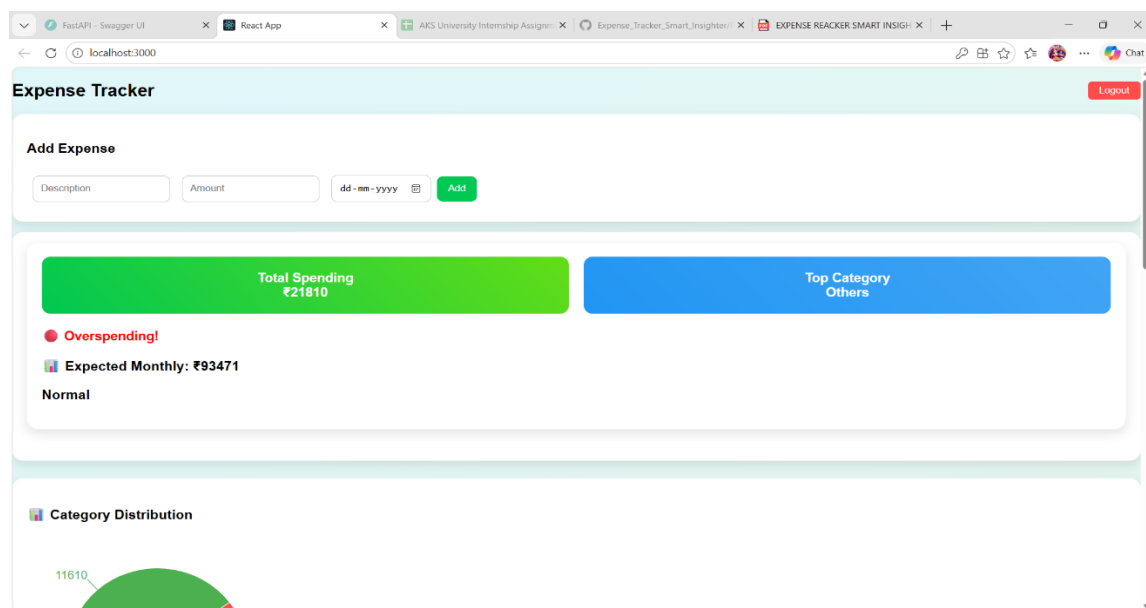
2. Dashboard:

The Dashboard is the application's central interface and provides a comprehensive overview of the user's financial data.

It displays key information such as:

- Total expenses
- Budget status
- Alerts and insights
- Summary of recent transactions

The dashboard is designed to give users a quick understanding of their spending behaviour. Information is presented in a structured and visually clear manner, making it easy to interpret. This screen acts as the control centre where users can monitor and manage their finances efficiently.



3. Add Expense:

The Add Expense section allows users to record new financial transactions.

Users can input details such as:

- Expense amount
- Description of the transaction

Once submitted, the system processes the input and automatically assigns a category based on the description using built-in logic. The form is designed to be minimal and easy to use, ensuring that users can quickly add expenses without unnecessary complexity.

This feature is essential for maintaining accurate and up-to-date financial records.

4. Expense List:

The Expense List displays all recorded transactions in a structured format.

Each entry includes:

- Amount
- Description
- Category
- Date (if implemented)

Users are provided with options to edit or delete existing records, giving them full control over their data. This section ensures transparency and allows users to review and manage their expenses effectively.

The layout is organised to support easy scanning and interaction.

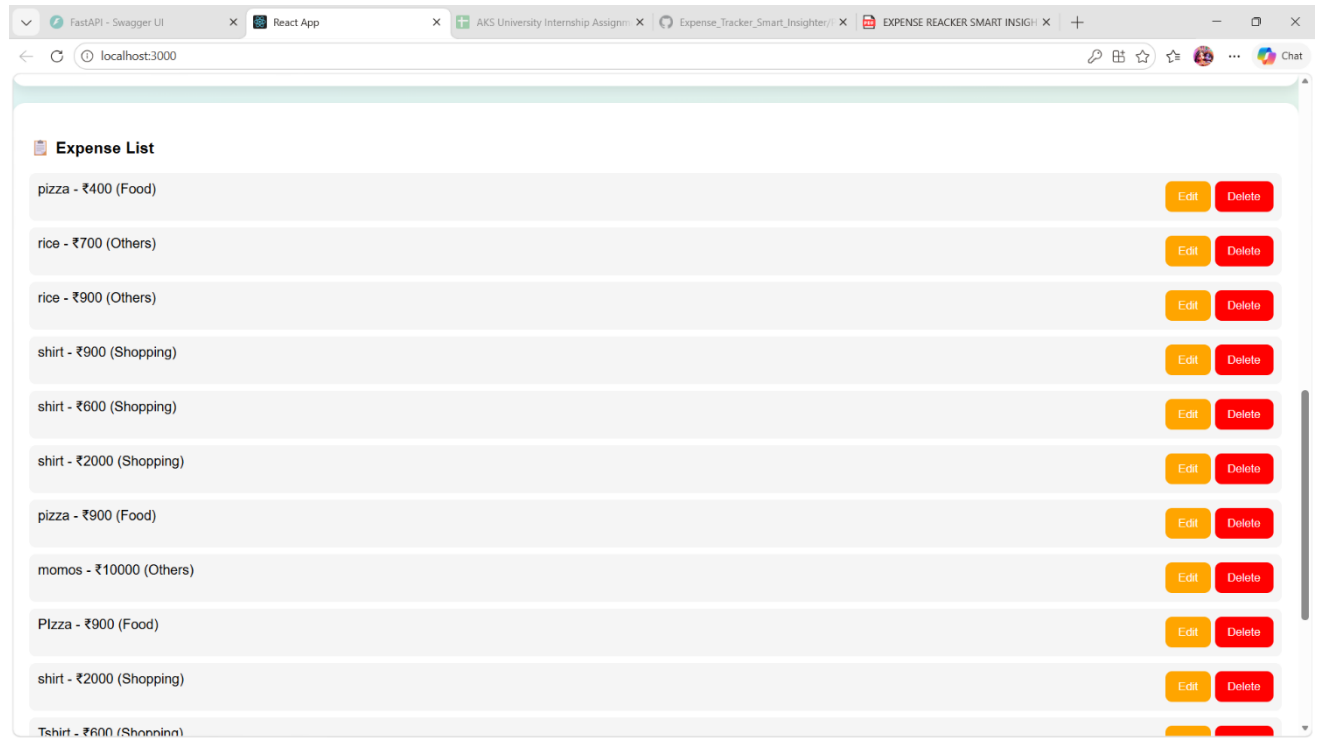


Figure : Expense List

5. Data Visualisation:

The application includes visual representations of financial data to improve understanding and analysis.

Key visual elements include:

- **Pie Chart** showing category-wise distribution of expenses.

These charts transform raw data into meaningful insights, helping users quickly identify patterns and make informed decisions. The visualisations are integrated into the dashboard for easy access.

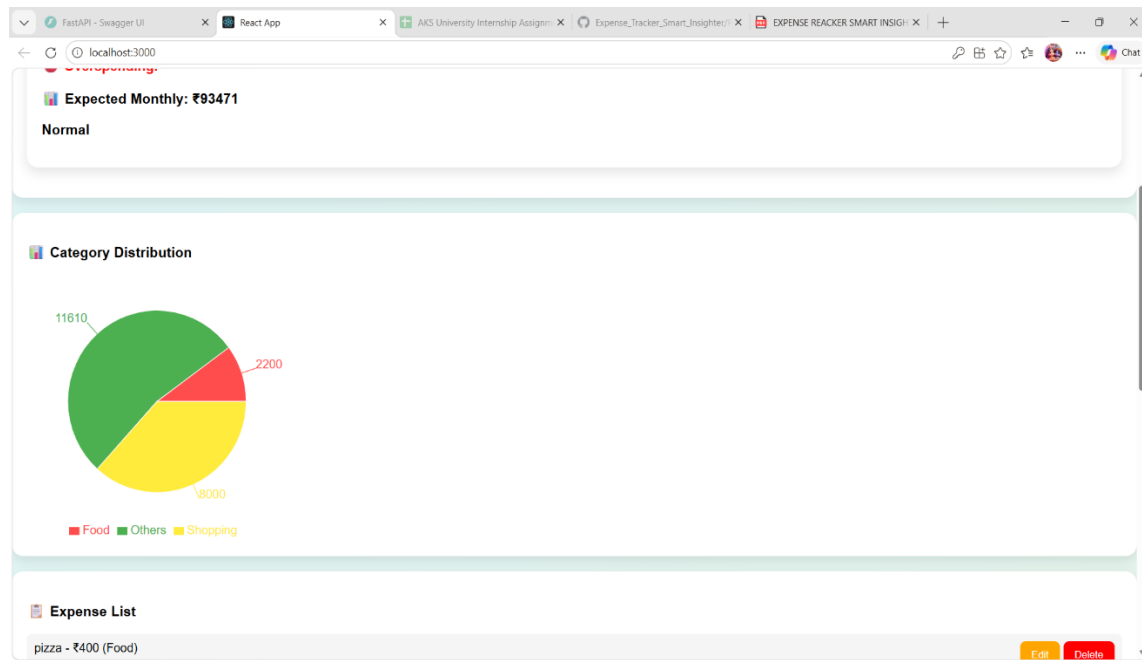


Figure :Visual Analytics

CONCLUSION

The **Expense Tracker with Smart Insighter** project successfully demonstrates the design and development of a full-stack web application that goes beyond basic expense tracking by incorporating intelligent analytical features. The system enables users not only to record and manage their daily expenses but also to gain meaningful insights into their financial behaviour, making it both practical and value-driven. Throughout the development process, modern technologies such as **React (frontend)**, **FastAPI (backend)**, and **PostgreSQL (database)** were effectively integrated to build a responsive, efficient, and scalable application. The modular architecture ensured clear separation of concerns, which contributed to better maintainability and easier future enhancements. Key features such as automatic expense categorisation, predictive spending analysis, budget monitoring, and anomaly detection significantly enhance the functionality of the system. These features transform raw transactional data into actionable insights, helping users understand spending patterns and make informed financial decisions.

The inclusion of an interactive dashboard and visual elements, such as charts and summaries, improves user engagement and simplifies data interpretation. This makes the system not only functional but also user-friendly and intuitive. From a learning perspective, the project provided hands-on experience in full-stack development, API design and integration, database modelling, and implementation of basic AI-driven logic. It also strengthened understanding of real-world system design, problem-solving approaches, and the importance of building scalable applications.

In conclusion, the project successfully achieves its intended objectives and demonstrates a strong balance between functionality, usability, and intelligent features. It also establishes a solid foundation for future improvements, such as integrating advanced machine learning models, enhancing predictive accuracy, and deploying the system on cloud platforms for real-world usage.